CS251 SOFTWARE ENGINEERING 1

[MAINSTREAM PROGRAMME]

COURSE PROJECT INSTRUCTIONS SPRING 2026

Instructions to Students:

This is a group work project. Each group in *the* **Mainstream Programme** consists of **6 to 7 students**. The TA(s) must approve the group members through registration. Each group must develop the assigned idea using **Object-Oriented Technologies** & PHP. *Please note that every idea (from the provided list) can be assigned to a limited number of teams*.

Project Objectives: The objectives of this project can be summarized in the following:

- Eliciting, Analysing, Validating, Specifying, & Managing the Software Requirements.
- Using Software Engineering Processes and approaches, & UML diagrams in developing *(both designing & implementing)* an Object-Oriented-based application.
- Applying design & architectural patterns.
- Testing.

Submission: Submission is done according to the following schedule:

- **Week 06:** Registering your team.
- **Week 07:** Ideas assigned to the teams.
- **Week 12/13:** Discussion *(detailed schedule TBA later)*.

Assessment: Assessment will include reports, code submitted, and discussions with team members. All team members must contribute across all phases, and each member's role must be clearly stated in each report. The Project will be assessed based on the following criteria:

- The number of functionalities your project covers (determines the complexity of your project).
- The quality of your documentation.
- The correctness of your analysis and design diagrams.
- Design Quality: using patterns, levels of abstractions, etc.
- Implementation correctness.

Feedback: The teaching assistants will provide feedback to each group upon submission of each phase. Further details could be provided during office hours with the teaching assistants.

Academic Integrity: You can only submit your work. Any student suspected of plagiarism will be subject to the procedures set out by the Faculty *(including being marked as failing the course)*. Capital University's (*formerly* Helwan University's) policies on academic integrity will be enforced against students who violate the University's standards of academic integrity. Examples of behaviour that is not allowed are:

- Copying all or part of someone else's work and submitting it as your own;
- Giving another student in the class a copy of your work,
- Copying parts from the internet, textbooks, etc.
- If you have any questions concerning what is allowed, please don't hesitate to discuss them with me.

CS251 SOFTWARE ENGINEERING 1 SPRING 2026

COURSE PROJECTS' DESCRIPTIONS (22 IDEAS)

Contents

Important Notes for Groups [Read Carefully] 3
1) A Peer-to-Peer (P2P) Specialized Tool Library 4
2) Sustainable Fashion & Upcycling Marketplace 6
3) Remote Freelance "Gig" Platform for Specialized Services 9
4) Smart Pet Care & Veterinary Management System 12
5) Community-Driven Disaster Relief Coordination 15
6) Collaborative Itinerary & Group Travel Planner 18
7) Intellectual Property (IP) & Patent Tracking System 21
8) Virtual Career Fair & Networking Hub 24
9) Smart Home Energy & Resource Monitor (Simulation) 27
10) Eco-Tourism & Local Experience Booking Portal 29
11) Smart Urban Parking Management System 32
12) A Subscription Box Customization & Delivery Portal 35
13) Integrated Community Garden & Allotment Manager 38
14) Independent Bookstore & Reader's Club Network 41
15) Small-Scale Logistics & Fleet Tracking Platform 44
16) Holistic Mental Health & Wellness Portal 47
17) Virtual Laboratory & Research Resource Scheduler 50
18) Hyper-Local Crowdfunding for Community Projects 53
19) Boutique Hotel & Hospitality Management Suite 56
20) Senior Care & Neighborly Assistance Network 59
21) AI-Driven Smart Recruitment & Interview Management System 62
22) IoT-Integrated Smart Warehouse & Inventory Logistics Portal 65

Important Notes for Groups [Read Carefully]

1) The descriptions are indicative and not final; thus, you need to do your refinements to these requirements by adding your assumptions and constraints. You should rely on an existing application that performs functional requirements similar to the ones you're developing. For many of the ideas, I've indicated an application you might model on.
2) Some of the ideas are well defined and detailed, while others are NOT. For ideas that are not fully detailed, you need to elaborate and explain how things work.
3) Finally, feel free to discuss your refinements with your TAs during the labs.
4) In case one (or more) of the functionalities within the application requires an advanced algorithm, just simulate that function using a very basic algorithm. That is, for any advanced algorithms (such as route optimization, automated recommendations, geospatial search, or trust-score calculation), students should simulate the function using a basic algorithm. The assessment will prioritize the correctness of analysis and design diagrams and the quality of documentation.
5) In the design phase, you're expected to design at least 5 non-CRUD functions per team member (that is, a team of 6 members should include at least 30 functions, while a team of 7 members should include at least 35). In addition to designing the simple CRUD (Create, Read, Update, Delete) operations.
6) In the implementation phase, you're expected to implement at least 4 functions per team member (that is, a team of 6 members should implement at least 24 functions, while a team of 7 members should implement at least 28). In addition, you are also expected to implement the simple CRUD (Create, Read, Update, Delete) operations.

N.B. We understand that you might positively refer to AI tools to support your research. Please note that, in case there are any concerns about AI-generated submission content, you will be invited for an opportunity to verify and defend your work.

17) Virtual Laboratory & Research Resource Scheduler

[Example(s): https://www.stratocore.com/ | https://www.ilab.agilent.com/]

A management system for university labs where researchers can book high-end equipment (e.g., microscopes, servers) and track research grants.

- **Key Features:** Equipment "Interlock" logic (preventing double booking), grant-based billing, safety certification tracking, and automated reporting on equipment utilization.

Detailed Project Overview: This is a high-precision resource management ecosystem designed for university research environments and core facilities. The platform manages the logistical "heart" of scientific research by coordinating access to expensive, high-end equipment (e.g., Electron Microscopes, DNA Sequencers, High-Performance Computing Clusters). The system moves beyond simple booking by integrating Grant-Based Financial Accounting, Safety Compliance Enforcement, and Hardware Interlock Logic. LabSync ensures that only qualified researchers can access specific tools and that all usage is accurately billed to the correct research grants, providing administrators with deep insights into equipment ROI and utilization.

Core Modules

1. **Resource & Interlock Module:** Manages physical equipment states, technical specifications, and hardware access logic.
2. **Research & Grant Accounting Module:** Handles the complex mapping of research projects, funding sources, and consumption-based billing.
3. **Compliance & Certification Module:** Tracks user training, safety certifications, and specialized lab access permissions.
4. **Analytics & Utilization Module:** Provides institutional reporting on uptime, maintenance costs, and research throughput.

Design Functions (42 Functions)

A. Equipment & Interlock Logic (11 Functions)

1. **Multi-State Hardware Interlock:** A state-machine logic that prevents a user from "Starting" a session unless they have an active reservation and valid safety credentials.
2. **Sequential Booking Dependency:** Logic that ensures a secondary piece of equipment (e.g., a Cooling Unit) is automatically reserved if the primary equipment (e.g., a Laser) requires it.
3. **Real-Time Session Heartbeat:** A simulation logic that tracks "Active Usage" and automatically terminates sessions if the user does not check in, freeing the resource.
4. **Emergency "Maintenance Lockout":** A high-priority function that instantly cancels all future reservations and notifies users when a machine breaks down.
5. **Technical Specification Mapping:** An OO approach to storing diverse equipment metadata (e.g., Voltage requirements for a physical tool vs. RAM/Cores for a server).
6. **Usage-Based Calibration Trigger:** Automatically flagging equipment for calibration after X hours of run-time or Y number of sessions.
7. **Resource Over-Provisioning Guard:** Logic that limits the number of high-demand resources a single researcher can book simultaneously to ensure fair access.
8. **Power-Up/Cool-Down Buffer Logic:** Automatically inserting non-bookable "Buffer Blocks" between sessions to allow equipment to reach stable operating temperatures.
9. **Anomalous Consumption Detection:** Identifying if a machine is drawing more "Simulated Power" or "Consumables" than the standard profile suggests.

10. **Remote Hardware Simulation Bridge:** A logic gate allowing users to "Submit Jobs" to virtual resources (like a compute cluster) with priority queueing.
11. **Equipment "Health" Portfolio:** A permanent log of every repair, calibration, and software update performed on a specific asset.

B. Grant-Based Financials & Billing (11 Functions)

12. **Multi-Source Grant Partitioning:** Allowing a single research project to draw funds from multiple grants based on a pre-defined percentage split.
13. **Consumption-Based Rate Calculator:** Dynamic billing logic that applies different rates (Internal University Rate, External Partner Rate, Peak vs. Off-Peak).
14. **Grant "Hard-Cap" Enforcement:** Logic that prevents a user from booking equipment if their associated grant has fallen below a specific balance.
15. **Consumable Auto-Deduction:** Automatically billing the grant for "Perishables" (e.g., liquid nitrogen, specialized slides) used during a session.
16. **Indirect Cost (Overhead) Simulator:** Calculating and adding university administrative "Overhead" fees to every equipment transaction.
17. **Grant Expiry Logic:** Automatically flagging and preventing new charges to grants that have passed their "End Date."
18. **Principal Investigator (PI) Approval Workflow:** A logic gate where a PI must "Authorize" a specific researcher to spend funds from a grant.
19. **Bulk Invoice Generator:** Aggregating all usage for a specific lab group over a month into a single itemized PDF for the finance department.
20. **Disputed Charge Reconciliation:** A workflow for researchers to challenge a bill (e.g., "The machine failed mid-session") and for admins to issue credits.
21. **Grant Re-allocation Engine:** Moving a previously billed session from one grant to another for accounting corrections.
22. **Real-Time Spending Forecast:** An algorithm that projects a project's "Runway" based on current equipment consumption rates.

C. Safety, Training & Compliance (10 Functions)

23. **Prerequisite Certification Checker:** A logic gate that prevents a user from booking "Equipment B" unless they have completed the "Level 1 Training" for "Equipment A."
24. **Certification Expiry Tracker:** An observer-pattern implementation that notifies users 30 days before their "Lab Safety Certificate" expires.
25. **Supervised Session Logic:** A specialized booking type where a "Trainee" can only book equipment if a "Certified Supervisor" is also present.
26. **Digital Training Logbook:** An unalterable record of every training session attended by a researcher, signed off by a Lab Manager.
27. **Hazardous Material (HAZMAT) Tracker:** Mapping specific equipment to required safety gear (PPE) and triggering alerts if special disposal is needed.
28. **Incident & Injury Workflow:** A structured process for reporting equipment-related accidents, including automated "Lockout" of the involved user.
29. **Tiered Access Permissions:** Managing roles such as "Undergrad (Restricted)," "Ph.D. (Standard)," and "Staff (Full Access)."
30. **Guest Researcher Onboarding:** A time-limited state-machine for visiting scientists with temporary credentials and liability waivers.
31. **Security Clearance Verification:** Logic to ensure researchers meet specific security levels for sensitive "Dual-Use" technology.
32. **Automated Safety Briefing:** Forcing users to "Confirm Receipt" of the latest safety protocols before they can enter the "Live Session" interface.

D. Institutional Analytics & Administration (10 Functions)

33. **Utilization Heatmap Generator:** Visualizing which hours of the day and which specific tools are the "Bottlenecks" in the lab.
34. **Downtime Impact Report:** Calculating the "Lost Research Value" (simulated) whenever a high-end machine is out for repair.
35. **Equipment ROI Calculator:** Comparing the cost of maintenance and operation against the total hours of research produced.
36. **Publication & Output Linkage:** Allowing researchers to link specific "Research Papers" to the equipment used to generate the data.
37. **Role-Based Access Control (RBAC):** Managing permissions for "University Auditor," "Lab Manager," "Faculty PI," and "Researcher."
38. **Maintenance Resource Planner:** Coordinating the schedules of external technicians to minimize lab disruption.
39. **System Audit Trail:** A permanent, non-repudiable log of every booking, cancellation, and grant change for financial auditing.
40. **Multi-Facility Data Aggregator:** Allowing a university to view equipment stats across different departments (e.g., Physics vs. Biology).
41. **Database Integrity Manager:** Periodically archiving old session data and "Decommissioned" equipment records to maintain system speed.
42. **Automated Equipment Procurement Alert:** Notifying the university when a machine's "Maintenance-to-Usage" ratio suggests it is time to buy a replacement.